

Practical-9 OUTPUT

1. Code Implementation:

Shows the C program written to implement the Optimal Page Replacement algorithm, where the program checks future page references and replaces the page that will not be used for the longest time in upcoming requests.



```
GNU nano 7.2 optimal.c *
#include <stdio.h>

int main() {
    int frames = 3;
    int pages[] = {1,2,3,4,1,2,5,1,2,3,4,5};
    int n = 12;

    int frame[3];
    int pageFault = 0;
    int pageHit = 0;

    for(int i=0;i<frames;i++)
        frame[i] = -1;

    for(int i=0;i<n;i++){
        int found = 0;

        for(int j=0;j<frames;j++){
            if(frame[j] == pages[i]){
                found = 1;
                pageHit++;
                break;
            }
        }
    }

    ^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/_ Go To Line M-E Redo      M-G Copy
```

2. Code Logic (Page Checking):

Displays the part of the program where memory frames are checked to determine whether the requested page is already present in memory, which helps identify page hits and page faults.



```
GNU nano 7.2 optimal.c *
    }
}

if(found == 0){
    int pos = -1;

    for(int j=0;j<frames;j++){
        if(frame[j] == -1){
            pos = j;
            break;
        }
    }

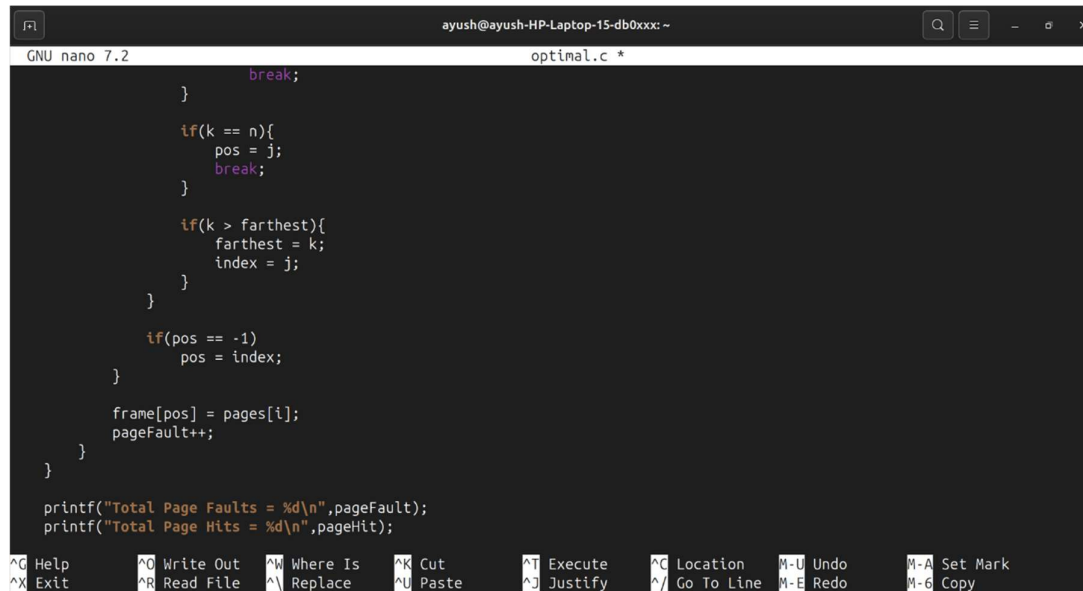
    if(pos == -1){
        int farthest = i;
        int index = -1;

        for(int j=0;j<frames;j++){
            int k;
            for(k=i+1;k<n;k++){
                if(frame[j] == pages[k])
                    break;
            }
        }
    }
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/_ Go To Line M-E Redo      M-G Copy
```

3. Replacement Decision:

Illustrates the logic used to determine which page should be replaced by analysing future page references and selecting the page that will be used farthest in the future.



```
GNU nano 7.2 optimal.c *
    }
    break;
}

if(k == n){
    pos = j;
    break;
}

if(k > farthest){
    farthest = k;
    index = j;
}
}

if(pos == -1)
    pos = index;
}

frame[pos] = pages[i];
pageFault++;
}

printf("Total Page Faults = %d\n",pageFault);
printf("Total Page Hits = %d\n",pageHit);
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^M-A Set Mark
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^/_ Go To Line ^E Redo       ^M-E Copy
```

4. Result Calculation:

Shows the section of the program where the total page hits, page faults, and their percentages are calculated after processing the entire page reference sequence.



```
GNU nano 7.2 optimal.c *
    }
    index = j;
}
}

if(pos == -1)
    pos = index;
}

frame[pos] = pages[i];
pageFault++;
}

printf("Total Page Faults = %d\n",pageFault);
printf("Total Page Hits = %d\n",pageHit);

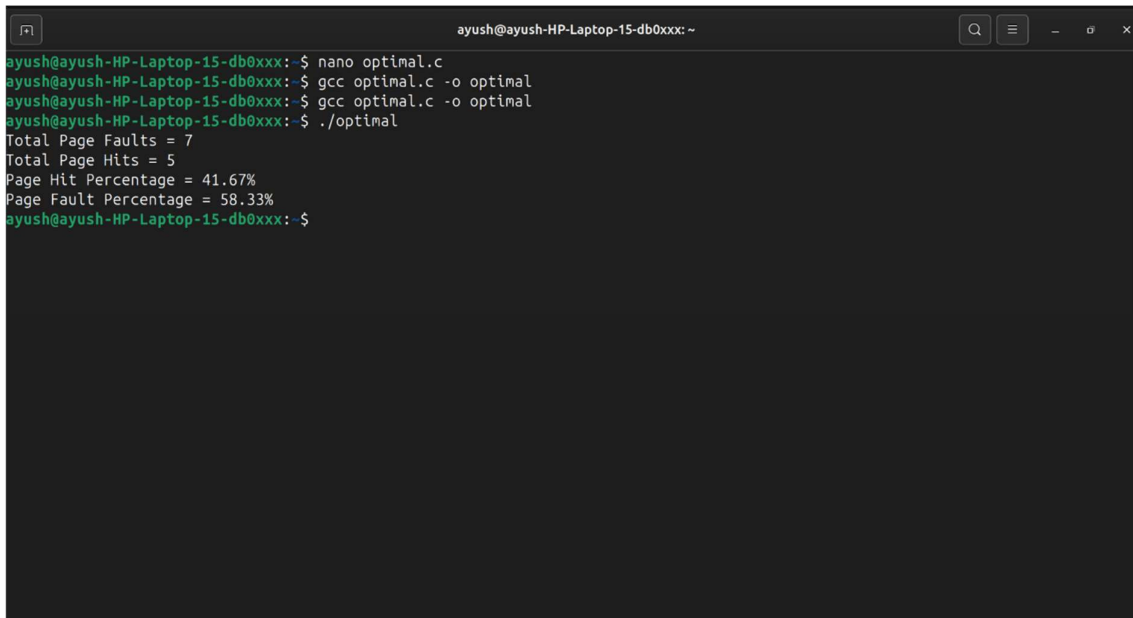
float hitPercent = (float)pageHit/n*100;
float faultPercent = (float)pageFault/n*100;

printf("Page Hit Percentage = %.2f%%\n",hitPercent);
printf("Page Fault Percentage = %.2f%%\n",faultPercent);

return 0;
}
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^M-A Set Mark
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^/_ Go To Line ^E Redo       ^M-E Copy
```

5. Program Output:

Displays the final execution results in the terminal, showing the total number of page faults, page hits, and the calculated hit and fault percentages using the Optimal page replacement algorithm.



```
ayush@ayush-HP-Laptop-15-db0xxx: ~  
ayush@ayush-HP-Laptop-15-db0xxx: $ nano optimal.c  
ayush@ayush-HP-Laptop-15-db0xxx: $ gcc optimal.c -o optimal  
ayush@ayush-HP-Laptop-15-db0xxx: $ gcc optimal.c -o optimal  
ayush@ayush-HP-Laptop-15-db0xxx: $ ./optimal  
Total Page Faults = 7  
Total Page Hits = 5  
Page Hit Percentage = 41.67%  
Page Fault Percentage = 58.33%  
ayush@ayush-HP-Laptop-15-db0xxx: $
```